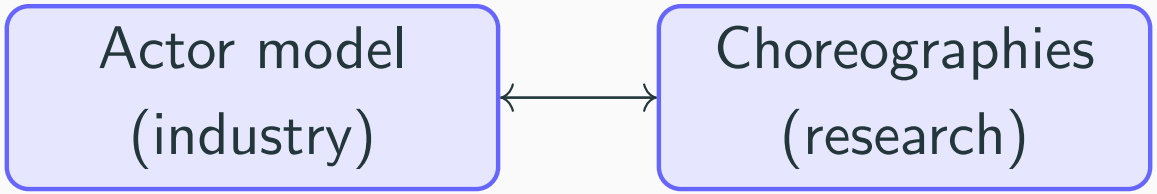
Choreographies for Program Understanding

Gabriele Genovese, Ivan Lanese, Cinzia Di Giusto, Emilio Tuosto, and Germán Vidal

Distributed Systems are Hard

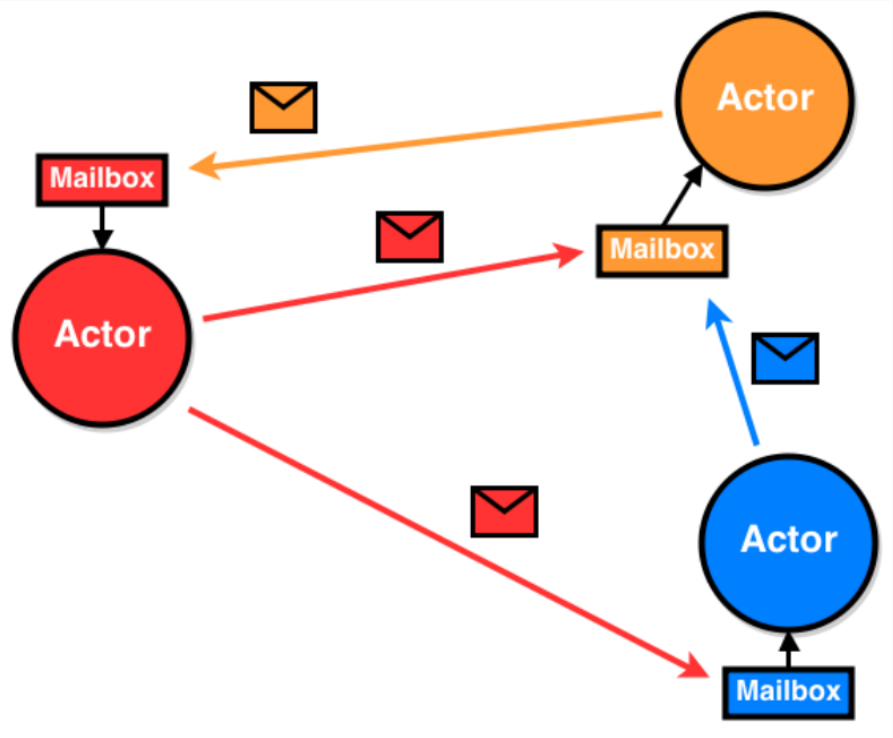
Abstractions in order to simplify:



The Actor Model

Main concepts

- Process with mailbox
- Asynchronous messaging



The Actor Model

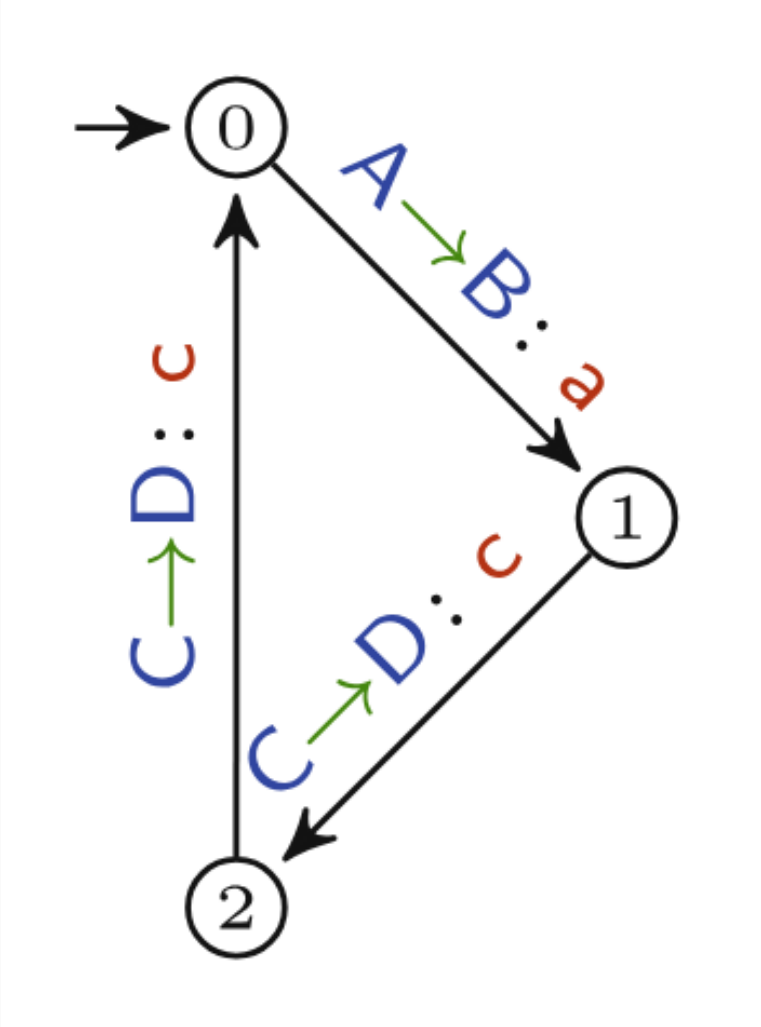
Ecosystem

- Erlang, Elixir, Scala
- Akka (Java), Actix (Rust)
- Go routines and channels

Choreography Automata [1]

Informally:

- Choreography: describes distributed protocols
- Paired with automata theory



[1] Barbanera et al. "Choreography automata." COORDINATION, 2020.

Global vs Local View

Global

The communication system is seen as a whole.

Local

An actor's individual perspective.

Deadlock

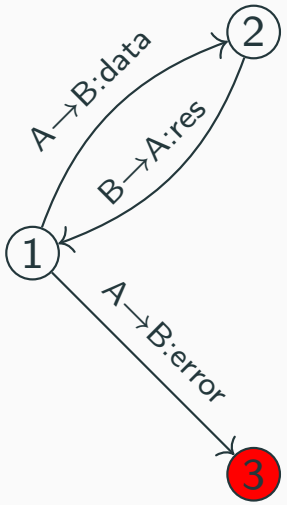
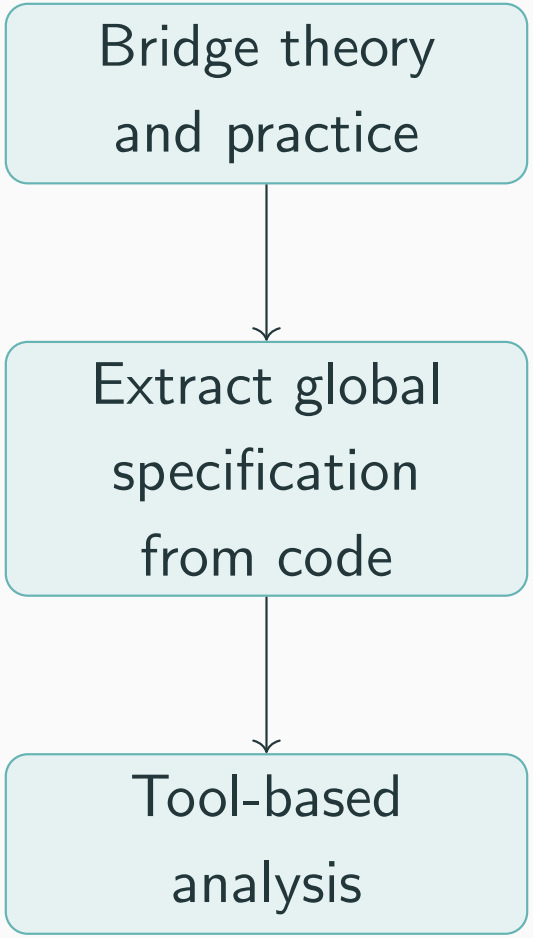


Figure 1: A deadlock can occur if a non-final state without outgoing transitions is present.

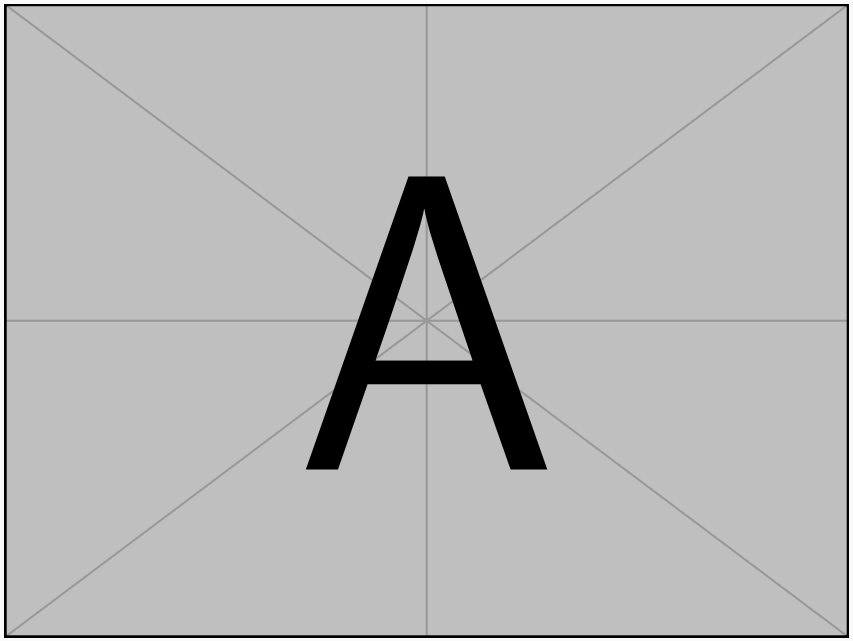
Aim


Bridge theory and practice
Extract global specification from code
Tool-based analysis

Why

Benefits

- Highlights bugs
- Improves understanding of the code
- Useful for legacy code
- Educational potential



Requirements and Problems

Requirements

- Bottom-up analysis
- Over-approximation
- Mainstream language support
- Simple notation

Problems

- Undecidability
- Complex models

Motivating Examples

Two example:

Dining Philosophers
(deadlock)

Bank Account
(race condition)

Dining philosophers example

```
philosopher(Fork1, Fork2) →
  send req to Fork1,
  receive ack from Fork1,
  send req to Fork2,
  receive ack from Fork2,
  eat(),
  send release to Fork1,
  send release to Fork2,
  philosopher(Fork1, Fork2).

fork() →
  receive req from Phil,
  send ack to Phil,
  receive release from Phil,
  fork().
```

Dining Philosophers global-view

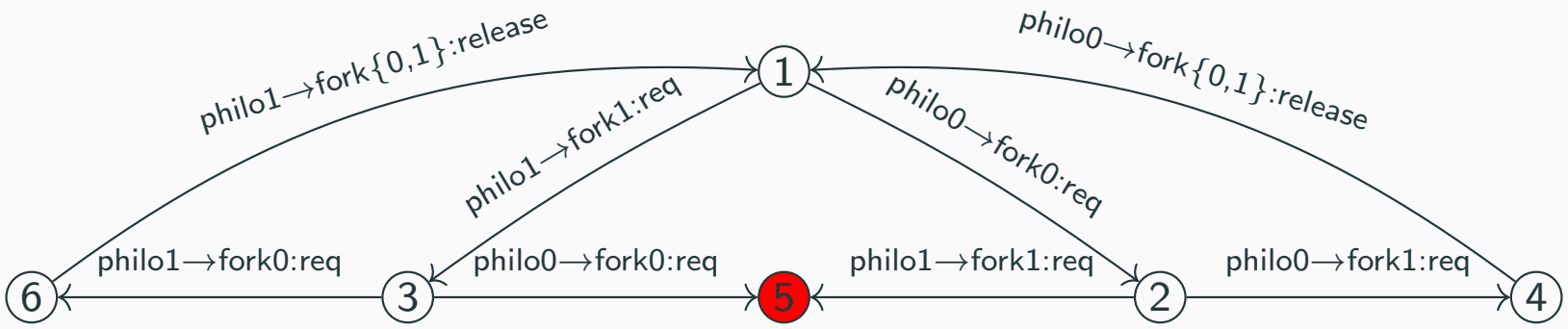


Figure 2: Global view of the dining philosophers example.

Bank account example

```
account(Value) →
  receive
    read from Client →
        send Value to Client,
        account(Value);
    NewValue from Client →
      account(NewValue).

client() →
  send read to Acc,
  receive Value from Acc,
  % operations on Value
  send NewValue to Acc.
```

Bank account global-view

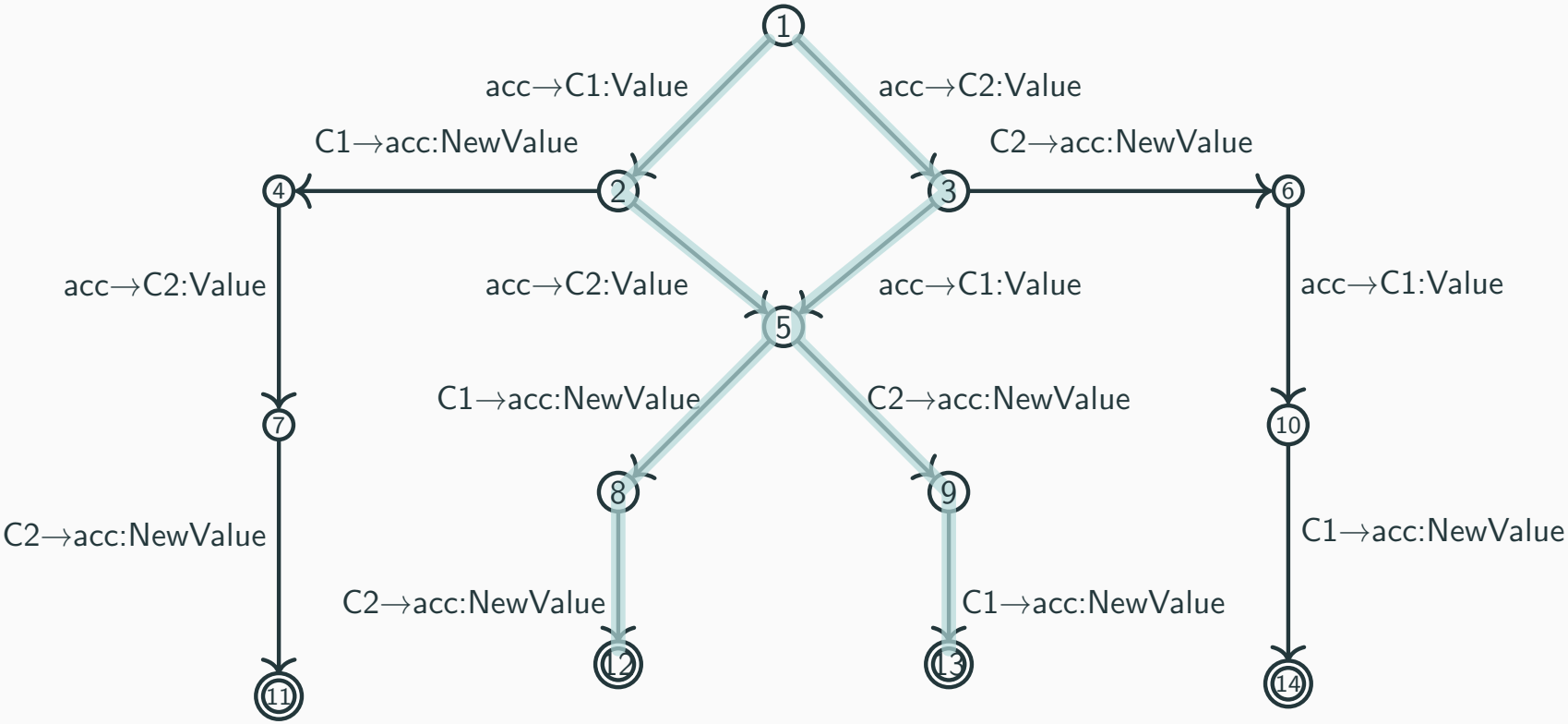


Figure 3: Global view of the bank account example

Conclusion

Summary

- Bridge the gap between programming and theoretical frameworks
- Use of bottom-up and over-approx techniques

Future Work

- WIP on a tool called Chorer

Thanks for listening!